

# STEAM: Self-Supervised Temporal Ensemble Advantage Modeling for Real-World Robot Learning

Anonymous Author(s)

Affiliation

Address

email

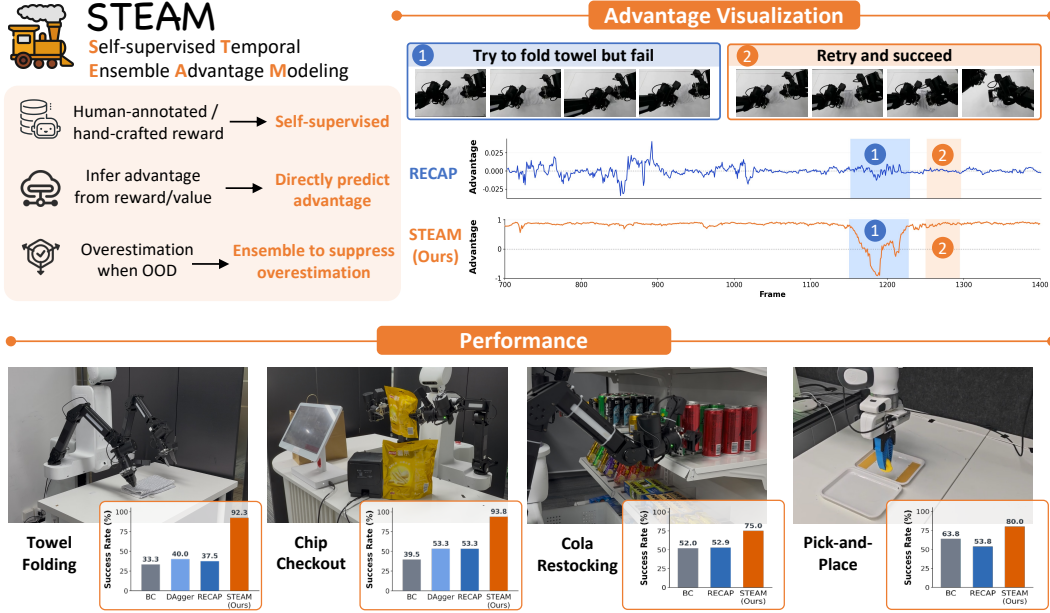


Figure 1: **STEAM**. STEAM is a self-supervised advantage modeling framework for real-world robot learning. It learns advantage prediction offline from expert demonstrations without manual annotations or hand-crafted rewards, and can be applied to expert data, human corrections, and policy rollouts to provide robust advantage estimates. When combined with CFGRL [1], STEAM substantially improves policy performance on various real-world tasks.

**Abstract:** Real-world robot learning increasingly relies on heterogeneous data, but demonstrations and rollouts often mix useful progress with stalls, corrections, and suboptimal behavior. Effective policy learning therefore requires frame-level advantages that distinguish reliable local progress from failures and regressions. We propose Self-supervised Temporal Ensemble Advantage Modeling (STEAM), a label-free method that learns such advantages from expert demonstrations. STEAM trains an ensemble of temporal-offset predictors on frame pairs within expert trajectories, using the normalized temporal offset between two frames as a self-supervised signal. Each predictor maps a frame pair to a distribution over temporal offsets, which is converted into a scalar advantage. STEAM then takes the minimum advantage across the ensemble to score mixed-quality rollout data conservatively. Across real-world bimanual towel folding, chip checkout, cola restocking, and single-arm pick-and-place tasks, STEAM identifies stalls, failures, and recoveries. When combined with CFGRL, STEAM

15 further improves policy success rate by 59%, 54.3%, 23% and 16.2% over base-  
16 lines, respectively.

17 **Keywords:** Advantage Modeling, Self-Supervised Learning, Robot Manipulation

## 18 1 Introduction

19 The rapid progress of robot foundation models relies heavily on large collections of expert demon-  
20 strations. However, collecting expert trajectories is expensive, and real robot datasets often contain  
21 mixed-quality behavior, especially in complex long-horizon tasks [2, 3, 4]. To broaden state-action  
22 coverage and improve robustness, recent methods [5, 6] incorporate non-expert trajectories, includ-  
23 ing policy rollouts, human interventions, and unsuccessful trials. These trajectories can be useful,  
24 but their quality often changes within the same episode. A rollout may make progress before fail-  
25 ing, and a human-intervention trajectory may contain poor autonomous behavior before recovery.  
26 Trajectory-level filtering can therefore discard useful transitions or keep harmful ones. This raises  
27 a central question for policy improvement: *how can we assign fine-grained credit to distinguish*  
28 *task-advancing actions from stalled or regressive ones?*

29 We formulate this question as frame-level advantage estimation. Estimating such advantages without  
30 dense rewards remains difficult. Recent robot-learning methods often follow a two-stage pipeline  
31 that first learns an intermediate reward or value signal [3, 5, 6, 7, 8, 9], then converts it into an  
32 advantage for weighted behavior cloning [3, 4, 10] or offline reinforcement learning [1, 11]. How-  
33 ever, deriving reliable signals remains difficult for three reasons. First, hand-crafted rewards, human  
34 annotations, and cross-trajectory calibration require substantial external supervision and limit scal-  
35 ability. Second, VLM-based reward or value models require broad pretraining and can still produce  
36 noisy frame-level signals when visual-language priors are not physically grounded. Third, many  
37 progress estimators impose an absolute or monotonic notion of progress, which is poorly matched  
38 to real rollouts that may temporarily regress, recover after mistakes, or leave the expert distribution.  
39 When a learned reward or value model is applied to these out-of-distribution rollout states, it may  
40 assign high advantages to unreliable transitions, creating false-positive training signals that drive  
41 policy optimization toward misleading behavior.

42 Instead of assigning an absolute progress value to each frame, we use the relative order of frames  
43 within the same expert trajectory. This relative signal is local to each demonstration and avoids  
44 cross-trajectory calibration or a hand-specified global progress curve. For any two frames, the nor-  
45 malized temporal offset can be computed directly from the trajectory. This yields dense pairwise  
46 progress supervision without hand-crafted rewards, human labels, or external value estimates.

47 Building on such motivations, we introduce **STEAM** (Self-supervised Temporal Ensemble  
48 Advantage Modeling), a label-free framework for frame-level advantage estimation. STEAM trains  
49 an ensemble of temporal-offset predictors on expert frame pairs. Each predictor outputs a distribu-  
50 tion over temporal bins, which is converted into a scalar advantage that reflects local temporal effi-  
51 ciency. STEAM uses reversed frame pairs from successful trajectories to expose regressive motion  
52 without failed demonstrations. To score mixed-quality trajectories robustly, STEAM takes the min-  
53 imum advantage across the ensemble, suppressing overestimated advantages on out-of-distribution  
54 samples. The resulting STEAM advantage can then be used to select high-advantage frames for pol-  
55 icy improvement with offline RL methods such as classifier-free guidance reinforcement learning  
56 (CFGRL) [1]. Our contributions are threefold.

- 57 • We formulate label-free frame-level advantage estimation from expert temporal structure, using  
58 normalized temporal offsets between frame pairs as dense self-supervised targets.
- 59 • We propose STEAM, which converts distributional temporal-offset predictions into scalar ad-  
60 vantages and uses conservative ensemble aggregation to suppress overestimated advantages on  
61 mixed-quality data.

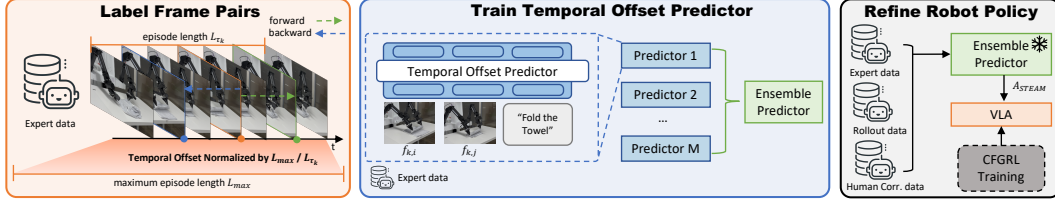


Figure 2: **STEAM framework.** (a) Expert demonstrations provide frame pairs for normalized temporal offset calculation. Both forward and reversed pairs are used as self-supervised targets. (b) An ensemble of  $M$  predictors is trained on expert data to map frame pairs and language instructions to categorical distributions over temporal bins, converting these into scalar advantage scores. (c) The trained ensemble scores mixed-quality data, and these  $A_{\text{STEAM}}$  advantages then guide a VLA policy through CFGRL for robot policy refinement.

- We validate STEAM on real-world bimanual towel folding, chip checkout, cola restocking, and single-arm pick-and-place tasks. STEAM localizes stalls, failures, and recoveries in demonstrations and rollouts, and when combined with CFGRL improves policy success rates by 59%, 54.3%, 23% and 16.2% over baselines, respectively.

## 2 Related Work

**Reward Models for Robot Manipulation.** Refining policies from mixed-quality data requires fine-grained signals, such as rewards, from which value and advantage estimates can be derived to distinguish useful behaviors from suboptimal ones [12, 11]. Since explicit rewards are rarely available in real-world robot manipulation, recent works instead learn reward or value estimates from auxiliary supervision, including human annotations [3, 4], hand-crafted reward functions [5, 6], and foundation-model priors [7, 13, 14, 15]. In contrast, STEAM predicts advantage directly rather than recovering it from a learned reward or value [5, 6]. This is similar to ARM [4], which estimates relative progress between frame pairs. The key difference lies in supervision: ARM relies on a classifier trained on manually annotated data, whereas STEAM exploits the temporal structure of expert demonstrations as self-supervision, requiring no manual annotation.

**Temporal Structure as Supervision Signals.** Temporal order is a free supervisory signal, since a frame’s position along a trajectory encodes task progress. This idea has long been used in self-supervised video representation learning [16, 17, 18, 19, 20]. In robot learning, temporal order is always converted into a dense reward signal to guide policy learning. TimeRewarder [8] regresses frame-wise temporal offsets into dense rewards. ReWiND [21] rewinds successful videos into failure-like reward sequences. VLAC [9] uses a large vision-language critic for signed progress deltas. Among existing methods, TimeRewarder is the closest prior work to STEAM, as both exploit signed temporal offsets between frame pairs, but STEAM uses them as per-frame advantages for offline data-quality assessment rather than dense rewards for online RL.

## 3 STEAM

STEAM trains an ensemble of temporal-offset predictors using expert demonstrations. These predictors then identify high-quality frames within heterogeneous, mixed-quality data to enhance policy performance (Figure 2). Section 3.1 details our self-supervised temporal offset learning target. Section 3.2 describes the signed-bin advantage predictor, which converts temporal offsets into robust scalar advantages. Section 3.3 integrates these advantages with classifier-free guidance reinforcement learning for effective policy optimization.

### 93 3.1 Normalized Temporal Offset as Learning Target

94 **Self-Supervised Temporal Targets.** Our training data comes exclusively from expert demonstra-  
 95 tions. Despite occasional brief stalls, the inherent temporal progression within these trajectories  
 96 reliably indicates task advancement, eliminating the need for hand-crafted rewards. For an expert  
 97 episode  $\tau_k = (f_{k,1}, f_{k,2}, \dots, f_{k,L_{\tau_k}})$ , we define the temporal offset between frames  $f_{k,i}$  and  $f_{k,j}$   
 98 as:

$$\Delta_{\tau_k}(i, j) = \Delta(f_{k,i}, f_{k,j}) = j - i. \quad (1)$$

99 Positive offsets, pairing a frame with a future observation, supervise forward progress. Conversely,  
 100 reversing expert trajectories provides negative offsets, creating pseudo-failure sequences that allow  
 101 the model to learn regressive behaviors from successful demonstrations alone.

102 **Trajectory-Length Normalization.** Raw temporal offsets are not directly comparable across tra-  
 103 jectories due to varying execution times. A given offset  $\Delta_{\tau_k}(i, j)$  might signify minor progress in  
 104 a long episode but substantial progress in a short one. To address this, we normalize the offset by  
 105 trajectory length:

$$\tilde{\Delta}_{\tau_k}(i, j) = \Delta(f_{k,i}, f_{k,j}) \cdot \frac{L_{\max}}{L_{\tau_k}} = (j - i) \cdot \frac{L_{\max}}{L_{\tau_k}}, \quad (2)$$

106 where  $L_{\max}$  is the maximum length among expert episodes (or a high quantile to reduce outlier sen-  
 107 sitivity). This normalization is crucial as it unifies temporal-offset targets from diverse trajectories  
 108 onto a consistent mathematical scale, ensuring stable supervision during training.

### 109 3.2 Distributional Temporal Offset Predictor

110 **Categorical Prediction and Bin Assignment.** We model the temporal-offset predictor as a dis-  
 111 tributional model. Given the long, high-frequency expert episodes, directly predicting continuous  
 112 offsets would lead to an unmanageably large output space. To address this, we bound the con-  
 113 tinuous temporal offset  $\tilde{\Delta}_{\tau_k}(i, j)$  within  $[-\tilde{\Delta}_{\max}, \tilde{\Delta}_{\max}]$  and discretize it into  $N$  evenly spaced  
 114 bins. This quantized offset forms a one-hot target vector  $\tilde{\Delta}_{\tau_k}^{\mathcal{B}}(i, j)$  of length  $N$ . Our predictor,  
 115  $p_{\theta}(\Delta \mid f_{k,i}, f_{k,j}, \ell)$ , parameterized by  $\theta$ , maps an expert frame pair  $(f_{k,i}, f_{k,j})$  and its natural lan-  
 116 guage instruction  $\ell$  to a categorical distribution over these  $N$  temporal bins. We train this predictor  
 117 by minimizing the cross-entropy loss:

$$\mathcal{L}(\theta) = \mathbb{E}_{\tau_k, i, j} \left[ H(\tilde{\Delta}_{\tau_k}^{\mathcal{B}}(i, j), p_{\theta}(\Delta \mid f_{k,i}, f_{k,j}, \ell)) \right], \quad (3)$$

118 where  $H$  denotes the cross-entropy between the one-hot target and the predicted categorical distri-  
 119 bution.

120 **Advantage Modeling.** To quantify a frame’s contribution to task progress, we derive a scalar  
 121 advantage from the learned distributional predictor. Unlike standard RL advantages, our metric  
 122 focuses solely on temporal efficiency. For a frame  $f_{k,i}$ , we infer the model’s prediction using a  
 123 future frame  $f_{k,i+H}$  (fixed lookahead  $H$ ). We then collapse the predicted categorical distribution  
 124 into an expected bin index. The advantage score is computed as the normalized expected bin index  
 125 minus the ground-truth quantized temporal offset:

$$A(f_{k,i}; \theta) = \frac{2}{N} \left[ \mathbb{E}_{b \sim p_{\theta}(\cdot \mid f_{k,i}, f_{k,i+H}, \ell)} [b] - \tilde{\Delta}_{\tau_{max}}^{\mathcal{B}}(i, i + H) \right]. \quad (4)$$

126 Since  $H$  is constant and  $\tau_{max}$  refers to the episode with the largest length,  $\tilde{\Delta}_{\tau_{max}}^{\mathcal{B}}(i, i + H)$  is a de-  
 127 terministic offset. This formulation inherently prioritizes execution efficiency due to the trajectory-  
 128 length normalization (Eq. (2)). Shorter, more efficient expert demonstrations, scaled by a factor  
 129 greater than one, yield higher advantage scores, while slower, suboptimal executions are penalized.

130 **Ensemble for Overestimation Control.** A single predictor trained on expert data can become  
 131 overly confident on out-of-distribution rollout samples, leading to severely overestimated advantage  
 132 scores [22]. Such overestimations generate false-positive signals that degrade policy refinement. To





Figure 3: **Robot setup and tasks.** We evaluate STEAM on four real-world manipulation tasks with varying horizons: towel folding (5 stages), chip checkout (8 stages), and cola restocking (4 stages) using an ARX dual-arm robot, and pick-and-place (2 stages) using a single Franka arm. To train and evaluate STEAM, we collect datasets containing varying mixtures of expert demonstrations, autonomous rollouts, and human correction episodes. Detailed manipulation procedures and the dataset composition for each task is summarized in Appendix C.

mitigate this, we employ a conservative ensemble strategy. We train  $M$  independent temporal offset predictors  $\Theta = \{\theta_1, \dots, \theta_M\}$  with distinct random initializations using Equation (3). Our final metric, the STEAM advantage, is the minimum predicted advantage across the ensemble:

$$A_{\text{STEAM}}(f_{k,i}; \Theta) = \min_{m=1, \dots, M} A(f_{k,i}; \theta_m). \quad (5)$$

This minimum-aggregation exploits the tendency of ensemble members to agree within the training distribution but diverge in unfamiliar state spaces [22]. By penalizing high variance, this acts as a robust regularizer against reward overoptimization, a common issue in RLHF [23]. While this conservative bias might slightly reduce recall, it crucially suppresses false positives, which our empirical validation (Section 4.3) shows is vital for stable and effective policy optimization.

### 3.3 Policy Training with $A_{\text{STEAM}}$

After training the ensemble of temporal-offset predictors, we leverage  $A_{\text{STEAM}}$  as a frame-level advantage estimator for heterogeneous training data, which includes expert demonstrations, policy rollouts, and human corrections. STEAM assigns an advantage  $A_{\text{STEAM}}$  to each frame. We convert these advantages into binary optimality labels  $o_{k,i}$  for policy training. Recognizing that different data sources possess distinct advantage distributions, we apply quantile thresholding separately to each. Specifically, for a frame  $f_{k,i}$ , its optimality label is:

$$o_{k,i} = \mathbf{1}[A_{\text{STEAM}}(f_{k,i}; \Theta) \geq \delta_q], \quad (6)$$

where  $\delta_q$  represents the  $q$ -quantile threshold of STEAM advantages, dynamically chosen based on the data source. This label  $o_{k,i}$  indicates whether a frame signifies high-quality local progress under the STEAM advantage. We then integrate these labels as the optimality condition within CFGRL [1], guiding the policy to prioritize action generation towards samples with higher estimated local progress. Further details on CFGRL training and inference are provided in Appendix A.

## 4 Experiments

We evaluate STEAM’s effectiveness on real-world robot manipulation tasks, focusing on its ability to identify high-quality frames and improve policy performance. As illustrated in Figure 3, we evaluate on four real-world manipulation tasks. Across all tasks, each STEAM advantage predictor utilizes SigLIP-SO400M as the visual encoder and Gemma-3-270M as the language backbone, followed by task-specific prediction heads. Policy learning leverages  $\pi_0$  [24] as the policy backbone, which is integrated with STEAM via CFGRL. Further hyperparameters and training configurations are detailed in Appendix D. Our experiments address three key questions: Q1: Can STEAM distinguish high- and low-quality frames? Q2: Does STEAM help improve policy performance? Q3: How do the STEAM design choices affect final performance?

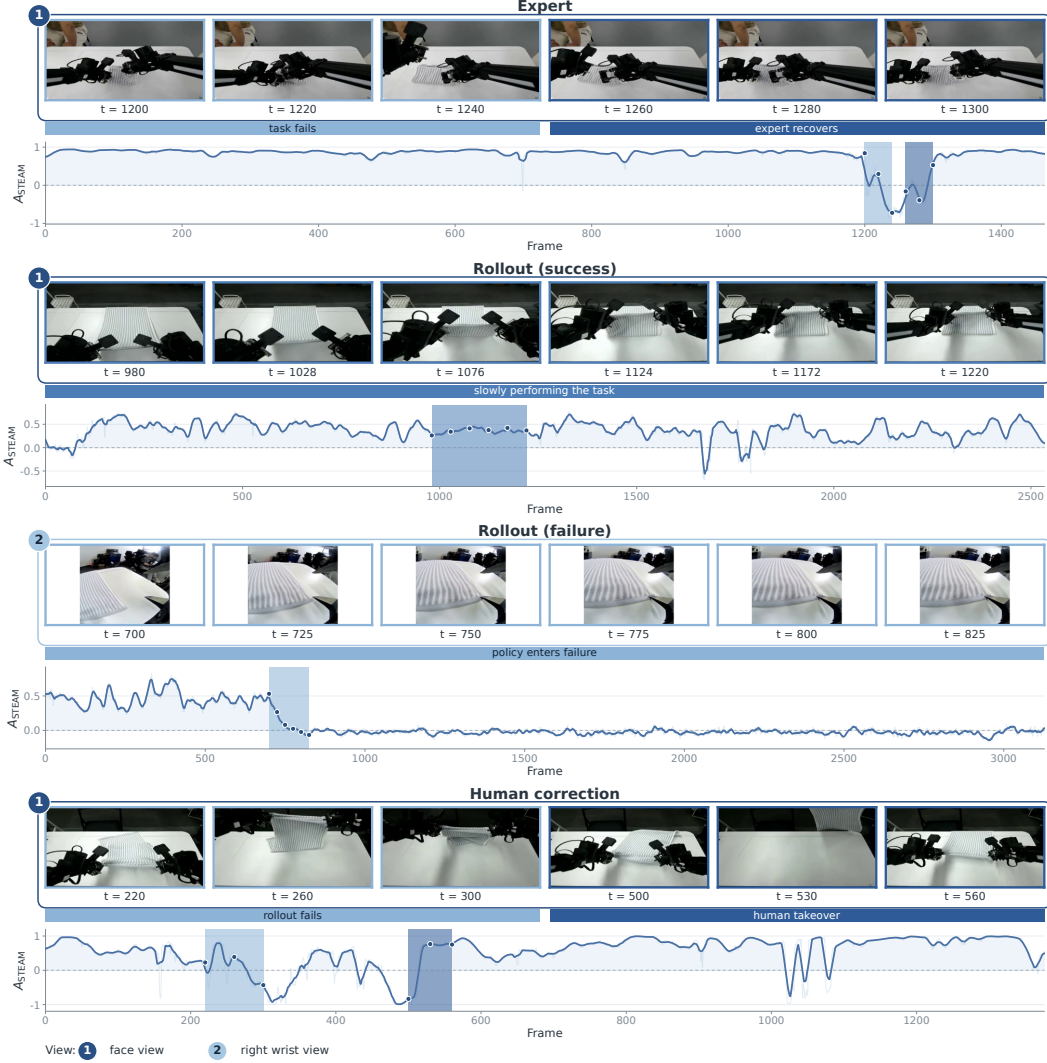


Figure 4: **Visualization of STEAM advantage curves on the towel folding task.** Frame-level  $A_{STEAM}$  is visualized on four representative episode types, including expert demonstrations, successful rollouts, failed rollouts, and human correction episodes. Images on top show corresponding frames from each episode, while shaded regions highlight segments with retry, slow progress, failure, or human takeover.

#### 163 4.1 STEAM Can Distinguish Task-advancing Actions

164 An effective advantage signal must reflect the nuances of task execution: remaining high during  
 165 proficient segments, decreasing during hesitation or failure, and recovering upon effective progress.  
 166 Figure 4 visualizes the STEAM advantage curves across four representative episode types in the  
 167 towel folding task: expert demonstrations, successful rollouts, failed rollouts, and human correction  
 168 episodes. For expert demonstrations, the advantage values remain consistently high throughout the  
 169 majority of the episode, experiencing only minor and transient drops around frames that involve neces-  
 170 sary retries or slight adjustments. Successful rollouts show lower and more fluctuating advantages  
 171 than expert demonstrations, reflecting slower yet still effective task execution. In contrast, failed  
 172 rollouts rapidly drop to near-zero advantage after entering failure states, where the policy becomes  
 173 stuck without recovering. Human correction episodes present an initial drop similar to failures, but  
 174 critically, show a clear recovery of advantage after human intervention. These results demonstrate  
 175 that STEAM provides a fine-grained, frame-level assessment capable of distinguishing proficient ex-

ecution from hesitation, failure, and recovery within a single episode. More advantage visualization results for the other three tasks are provided in Appendix E.

We further evaluate whether STEAM distinguishes data quality from the perspective of task progress. Figure 5 presents the probability density of frame-level  $A_{\text{STEAM}}$  across different data sources for all four tasks. Consistent trends are observed across all domains: expert demonstrations exhibit a strong concentration near +1, indicating that most frames correspond to high-quality execution that contributes directly to task completion. Successful rollouts systematically shift toward lower positive values, reflecting slower or more cautious execution. In contrast, failed rollouts place a prominent density peak around or below zero, which corresponds to the stagnation and lack of progress after a failure occurs. Human-correction episodes exhibit a broader, more distributed profile, combining the lower positive advantages from slow autonomous phases with the higher advantages recovered immediately after human takeover.

Figure 6 reports the episode-wise sum of frame-level advantages as an aggregate measure of overall task progress. While successful episodes yield comparable cumulative advantages, failed rollouts are clearly separated by significantly lower sums. Notably, cumulative advantages are higher for towel folding and chip checkout than for cola restocking and pick-and-place, particularly among failures. This discrepancy stems from their failure modes: towel and chip failures typically occur in middle or late stages, allowing positive advantages to accumulate early on, whereas cola and pick-and-place failures occur during the initial grasp, preventing any advantage accumulation. These quantitative results demonstrate that  $A_{\text{STEAM}}$  consistently provides a fine-grained assessment of task progress that reflects execution quality across diverse data sources and horizons. Furthermore, the clear separation in cumulative advantages suggests that STEAM can also, to some extent, be leveraged to discriminate between successful and failed trajectories at the episode level.

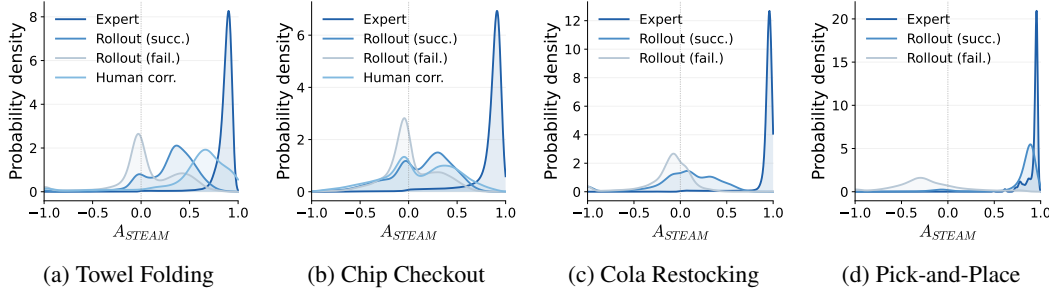


Figure 5: Probability density of frame-level  $A_{\text{STEAM}}$  across data types.

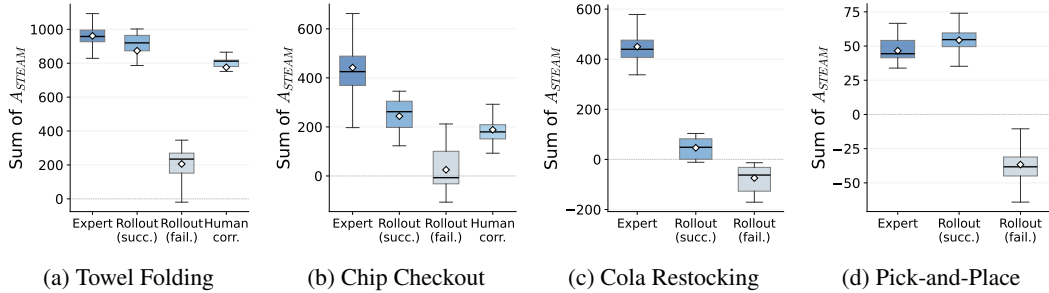


Figure 6: Sum of frame-level  $A_{\text{STEAM}}$  over each episode across data types.

## 4.2 STEAM Enhances Policy Performance

We evaluate whether the advantages predicted by STEAM can improve real-world policy performance by comparing STEAM against three established baseline methods: **Behavior Cloning (BC)** trains on the expert dataset only, **HG-DaGger** [25] augments policy training with human correc-

203 tive interventions, and **RECAP** [5] is a VLM-based value estimation model. RECAP and STEAM  
 204 leverage the full data source (expert demonstrations, human corrections, and autonomous rollouts).  
 205 For a fair comparison, RECAP is integrated with the same CFGRL backbone and run for a single  
 206 iteration.

Table 1: **Policy performance comparison.** Succ. represents the average success rate (%), Score denotes the average number of completed sub-stages, and Thr. is the throughput measured in successful episodes per hour. Note that because the BC baseline achieves reasonable success rates on the cola restocking and pick-and-place tasks, we do not collect human corrective demonstrations and therefore HG-Dagger is not evaluated on them.

| Method    | Towel Folding             |            |           | Chip Checkout               |            |           | Cola Restocking         |          |           | Pick-and-Place            |            |            |
|-----------|---------------------------|------------|-----------|-----------------------------|------------|-----------|-------------------------|----------|-----------|---------------------------|------------|------------|
|           | Succ. (%)                 | Score      | Thr.      | Succ. (%)                   | Score      | Thr.      | Succ. (%)               | Score    | Thr.      | Succ. (%)                 | Score      | Thr.       |
| BC        | 33.3                      | 3.3        | 42        | 39.5                        | 4.6        | 16        | 52                      | 2.36     | 71        | 63.8                      | 1.5        | 230        |
| HG-Dagger | 40                        | 3.7        | 48        | 53.3                        | 6          | 22        | —                       | —        | —         | —                         | —          | —          |
| RECAP     | 55.6                      | 2.9        | 39        | 53.3                        | 5.33       | 24        | 52.9                    | 2.1      | 46        | 53.8                      | 1.5        | 161        |
| STEAM     | <b>92.3</b> $\uparrow 59$ | <b>4.9</b> | <b>58</b> | <b>93.8</b> $\uparrow 54.3$ | <b>7.5</b> | <b>48</b> | <b>75</b> $\uparrow 23$ | <b>3</b> | <b>90</b> | <b>80</b> $\uparrow 16.2$ | <b>1.8</b> | <b>254</b> |

207 As shown in Table 1, STEAM achieves the best performance across all evaluation tasks. Most  
 208 notably, on the two challenging long-horizon tasks—towel folding and chips checkout—STEAM  
 209 reaches success rates by 92.3% and 93.8% respectively, with progress scores close to the maximum  
 210 stage counts. Beyond boosting success rates and task progress, STEAM also increases execution  
 211 speed. For instance, in the towel-folding task, autonomous rollout trajectories are inherently slower  
 212 than expert demonstrations. RECAP fails to filter out these slow-progress frames during training,  
 213 resulting in a throughput that is even lower than that of BC. In contrast, STEAM effectively prunes  
 214 these stagnant, low-quality frames, increasing the throughput to 58 successful episodes/hour.

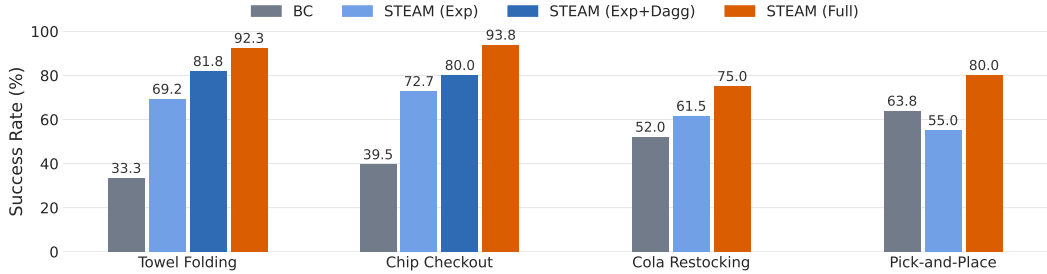


Figure 7: **STEAM performance across different training Data combinations.** We compare the success rate of Behavior Cloning against STEAM trained with varying sources of data: expert demonstrations only (STEAM (Exp)), expert data supplemented with human correction episodes (STEAM (Exp+Dagg)), and the full dataset containing expert, correction, and autonomous rollout data (STEAM (Full)).

215 To understand the impact of different data sources on policy learning, we conduct studies using  
 216 varying combinations of training data, as visualized in Figure 7. The results indicate that STEAM  
 217 can effectively filter high-quality frames from both human corrections and autonomous rollout data.  
 218 By leveraging these non-expert sources, STEAM broadens the policy’s state-action coverage, result-  
 219 ing in substantial improvements in success rate compared to training only on expert demonstrations.  
 220 However, in the expert-only setting (STEAM (Exp)), the performance compared to BC depends  
 221 on the task characteristics. For towel folding, chip checkout and cola restocking, STEAM (Exp)  
 222 improves success rates over BC (e.g., from 33.3% to 69.2% in towel folding), demonstrating that  
 223 STEAM can successfully isolate and emphasize the most critical, task-advancing segments within  
 224 expert demonstrations. Conversely, for pick-and-place, training on expert data only leads to a slight  
 225 decline compared to BC. This is primarily because pick-and-place represents a highly consistent,  
 226 short-horizon task where expert trajectories are already highly proficient, as reflected by the sharp  
 227 advantage concentration near +1 in Figure 5d. In such scenarios where expert data are already  
 228 clean but limited, the filtering mechanism may prune too many frames, effectively shrinking an

already small training set. Nevertheless, once autonomous rollouts and corrections are introduced (STEAM (Full)), the learned advantage signals become highly beneficial, successfully steering the policy back to superior performance by capitalising on the additional, imperfect trajectories.

### 4.3 Ablation Studies on STEAM Design Choices

We ablate the bin count  $N$  and ensemble size  $M$  on the towel-folding task with all expert demonstrations, human corrections, and rollout episodes. Table 2 reports the effect of bin count, and Table 3 reports the effect of ensemble size.

Table 2: Effect of bin count  $N$ .

| Bins               | Succ.       | Score      | Thr.      |
|--------------------|-------------|------------|-----------|
| $N = 2$            | 27.3        | 2.8        | 41        |
| $N = 8$            | 54.6        | 3.8        | 51        |
| $N = 32$ (default) | <b>92.3</b> | <b>4.9</b> | <b>58</b> |

Table 3: Effect of ensemble size  $M$ .

| Ens.              | Succ.       | Score      | Thr.      |
|-------------------|-------------|------------|-----------|
| $M = 1$           | 72.7        | 3.9        | 53        |
| $M = 3$ (default) | <b>92.3</b> | <b>4.9</b> | <b>58</b> |
| $M = 5$           | 90.9        | 4.6        | 55        |

**Effect of bin count.** The bin count  $N$  controls the granularity of the signed-bin temporal progress target. A small  $N$  reduces the target to a coarse forward/backward signal, while a larger  $N$  allows STEAM to distinguish different degrees of progress and regression. As shown in Table 2, increasing  $N$  improves performance, indicating that fine-grained temporal progress modeling provides a more useful advantage signal for policy learning.

**Effect of ensemble size.** The ensemble size  $M$  controls the strength of the worst-of- $M$  conservative aggregation strategy. When  $M = 1$ , STEAM uses a single predictor and cannot suppress overestimated advantages on out-of-distribution transitions. As shown in Table 3, increasing the ensemble size from  $M = 1$  to  $M = 3$  significantly improves the policy’s success rate from 72.7% to 92.3%, demonstrating that the ensemble-min estimator effectively reduces false-positive signals among high-advantage segments. However, performance does not improve further when scaling to  $M = 5$ . Therefore, we select  $M = 3$  as our default configuration. This choice balances performance and computational efficiency, as scaling the ensemble further introduces unnecessary training costs with diminishing returns. For a detailed qualitative analysis demonstrating how the ensemble suppresses advantage overestimation, please refer to Appendix E.4.

## 5 Limitations

STEAM learns advantages through temporal offsets, where frame pairs with the same offset receive the same target value. This ignores that different task phases may contribute unequally to progress. Structure-aware methods such as SPARS [26] assign different reward signals to different task stages, suggesting a possible way to incorporate phase-aware progress for fine-grained evaluation. Also, STEAM mainly relies on visual observations, which may fail to capture subtle but critical state differences. Adding robot state as a learning target could further expand STEAM’s ability to retrieve low-quality frames beyond visually observable errors.

## 6 Conclusion

We presented STEAM, a self-supervised advantage modeling framework that learns advantages from expert demonstrations without hand-crafted rewards or annotations. STEAM produces frame-level advantages that distinguish high- and low-quality samples and enable fine-grained credit assignment for policy learning. Experiments on four real-robot manipulation tasks show that STEAM can identify problematic samples in expert demonstrations and improve performance. When combined with rollout data, STEAM further improves policy learning by selecting useful samples from imperfect trajectories.



## References

- [1] K. Frans, S. Park, P. Abbeel, and S. Levine. Diffusion guidance is a controllable policy improvement operator. *arXiv preprint arXiv:2505.23458*, 2025. URL <https://arxiv.org/abs/2505.23458>.
- [2] S. Belkhale, Y. Cui, and D. Sadigh. Data quality in imitation learning. *Advances in neural information processing systems*, 36:80375–80395, 2023.
- [3] Q. Chen, J. Yu, M. Schwager, P. Abbeel, Y. Shentu, and P. Wu. SARM: Stage-aware reward modeling for long horizon robot manipulation. In *International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=aemqAxSc19>.
- [4] Y. Mao, Z. Yu, W. Mao, Y. Li, Q. Hu, Z. Lan, M. Zhu, and H. Chen. ARM: Advantage reward modeling for long-horizon manipulation. *arXiv preprint arXiv:2604.03037*, 2026.
- [5] A. Amin, R. Aniceto, A. Balakrishna, K. Black, K. Conley, G. Connors, J. Darpinian, K. Dhabalia, J. DiCarlo, D. Driess, et al.  $\pi_{0.6}^*$ : a VLA that learns from experience. *arXiv preprint arXiv:2511.14759*, 2025.
- [6] J. Yang, K. Lin, J. Li, W. Zhang, T. Lin, L. Wu, Z. Su, H. Zhao, Y.-Q. Zhang, L. Chen, P. Luo, X. Yue, and H. Li. RISE: Self-improving robot policy with compositional world model. *Robotics: Science and Systems*, 2026. URL <https://arxiv.org/abs/2602.11075>.
- [7] T. Lee, A. Wagenmaker, K. Pertsch, P. Liang, S. Levine, and C. Finn. RoboReward: General-purpose vision-language reward models for robotics. *arXiv preprint arXiv:2601.00675*, 2026. URL <https://arxiv.org/abs/2601.00675>.
- [8] Y. Liu, C. Wen, Y. Hu, D. Jayaraman, and Y. Gao. Timerewarder: Learning dense reward from passive videos via frame-wise temporal distance. *arXiv preprint arXiv:2509.26627*, 2025.
- [9] S. Zhai, Q. Zhang, T. Zhang, F. Huang, H. Zhang, M. Zhou, S. Zhang, L. Liu, S. Lin, and J. Pang. A vision-language-action-critic model for robotic real-world reinforcement learning. *arXiv preprint arXiv:2509.15937*, 2025. URL <https://arxiv.org/abs/2509.15937>.
- [10] H. Xu, X. Zhan, H. Yin, and H. Qin. Discriminator-weighted offline imitation learning from suboptimal demonstrations. In *International Conference on Machine Learning*, pages 24725–24742. PMLR, 2022.
- [11] I. Kostrikov, A. Nair, and S. Levine. Offline reinforcement learning with implicit q-learning. In *International Conference on Learning Representations*, 2022.
- [12] X. B. Peng, A. Kumar, G. Zhang, and S. Levine. Advantage-weighted regression: Simple and scalable off-policy reinforcement learning. *arXiv preprint arXiv:1910.00177*, 2019. URL <https://arxiv.org/abs/1910.00177>.
- [13] H. Tan, S. Chen, Y. Xu, Z. Wang, Y. Ji, C. Chi, Y. Lyu, Z. Zhao, X. Chen, P. Co, S. Xie, G. Yao, P. Wang, Z. Wang, and S. Zhang. Robo-Dopamine: General process reward modeling for high-precision robotic manipulation. *arXiv preprint arXiv:2512.23703*, 2025. URL <https://arxiv.org/abs/2512.23703>.
- [14] Y. J. Ma, J. Hejna, A. Wahid, C. Fu, D. Shah, J. Liang, Z. Xu, S. Kirmani, P. Xu, D. Driess, T. Xiao, J. Tompson, O. Bastani, D. Jayaraman, W. Yu, T. Zhang, D. Sadigh, and F. Xia. Vision language models are in-context value learners. In *International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=friHA15ofG>.
- [15] A. Liang, Y. Korkmaz, J. Zhang, M. Hwang, A. Anwar, S. Kaushik, A. Shah, A. S. Huang, L. Zettlemoyer, D. Fox, Y. Xiang, A. Li, A. Bobu, A. Gupta, S. Tu, E. Biyik, and J. Zhang. Robometer: Scaling general-purpose robotic reward models via trajectory comparisons. *Robotics: Science and Systems*, 2026. URL <https://arxiv.org/abs/2603.02115>.



- [16] D. Dwibedi, Y. Aytar, J. Tompson, P. Sermanet, and A. Zisserman. Temporal cycle-consistency learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1801–1810, 2019. URL [https://openaccess.thecvf.com/content\\_CVPR\\_2019/html/Dwibedi\\_Temporal\\_Cycle-Consistency\\_Learning\\_CVPR\\_2019\\_paper.html](https://openaccess.thecvf.com/content_CVPR_2019/html/Dwibedi_Temporal_Cycle-Consistency_Learning_CVPR_2019_paper.html).
- [17] K. Zakka, A. Zeng, P. Florence, J. Tompson, J. Bohg, and D. Dwibedi. XIRL: Cross-embodiment inverse reinforcement learning. In *5th Conference on Robot Learning*, 2022. URL <https://openreview.net/forum?id=R04DM85Z4P7>.
- [18] S. Nair, A. Rajeswaran, V. Kumar, C. Finn, and A. Gupta. R3M: A universal visual representation for robot manipulation. In *6th Conference on Robot Learning*, 2022. URL <https://arxiv.org/abs/2203.12601>.
- [19] Y. J. Ma, S. Sodhani, D. Jayaraman, O. Bastani, V. Kumar, and A. Zhang. VIP: Towards universal visual reward and representation via value-implicit pre-training. In *International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=YJ7o2wetJ2>.
- [20] Y. J. Ma, W. Liang, V. Som, V. Kumar, A. Zhang, O. Bastani, and D. Jayaraman. LIV: Language-image representations and rewards for robotic control. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, 2023.
- [21] J. Zhang, Y. Luo, A. Anwar, S. A. Sontakke, J. J. Lim, J. Thomason, E. Bıyık, and J. Zhang. ReWiND: Language-guided rewards teach robot policies without new demonstrations. In *9th Conference on Robot Learning*, 2025.
- [22] B. Lakshminarayanan, A. Pritzel, and C. Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [23] T. Coste, U. Anwar, R. Kirk, and D. Krueger. Reward model ensembles help mitigate overoptimization. In *International Conference on Learning Representations*, 2024. URL <https://arxiv.org/abs/2310.02743>.
- [24] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, et al.  $\pi_0$ : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [25] M. Kelly, C. Sidrane, K. Driggs-Campbell, and M. J. Kochenderfer. Hg-dagger: Interactive imitation learning with human experts. In *2019 International Conference on Robotics and Automation (ICRA)*, pages 8077–8083. IEEE, 2019.
- [26] R. He, Y. Wei, L. Yu, and X. Zeng. Spars: Structure-informed progress-aware reward shaping for fabric manipulation learning from demonstration. *Robotics and Autonomous Systems*, page 105499, 2026.

## A Classifier-Free Guidance RL Details

We integrate the frame-level advantages learned by STEAM into the classifier-free guidance reinforcement learning framework to guide policy optimization. Below we present the detailed training and inference procedures.

**CFGRL Training.** Once the STEAM advantage predictors are trained on the expert dataset, we compute the advantage score  $A_{\text{STEAM}}(f_{k,i}; \Theta)$  for each frame  $f_{k,i} \in \mathcal{D}_{\text{exp}} \cup \mathcal{D}_{\text{nexp}}$  using a fixed lookahead  $H$ . Here,  $\mathcal{D}_{\text{exp}}$  represents the expert demonstration dataset, and  $\mathcal{D}_{\text{nexp}}$  denotes the non-expert dataset, comprising both autonomous rollouts and human corrections. We then map these continuous advantages into binary optimality labels  $o \in \{0, 1\}$  by applying quantile thresholding separately to each data source:

$$o = \begin{cases} \mathbf{1}[A_{\text{STEAM}}(f_{k,i}; \Theta) \geq \varpi_q^{\text{exp}}], & f_{k,i} \in \mathcal{D}_{\text{exp}}, \\ \mathbf{1}[A_{\text{STEAM}}(f_{k,i}; \Theta) \geq \varpi_q^{\text{non-exp}}], & f_{k,i} \in \mathcal{D}_{\text{nexp}}, \end{cases} \quad (7)$$

where  $\varpi_q^{\text{exp}}$  and  $\varpi_q^{\text{non-exp}}$  denote the  $q$ -quantiles of  $\{A_{\text{STEAM}}\}$  computed over  $\mathcal{D}_{\text{exp}}$  and  $\mathcal{D}_{\text{nexp}}$ , respectively.

These binary optimality labels condition a flow-matching policy  $v_\phi(a_t, t, s, o)$  parameterizing a velocity field. During training, we construct a noisy action interpolant  $a_t$  at timestep  $t$  as:

$$a_t = (1 - t)a_0 + ta, \quad (8)$$

where  $a$  is the target action chunk from the dataset,  $a_0 \sim \mathcal{N}(0, I)$  is standard Gaussian noise, and  $t$  is sampled from a uniform distribution over  $[0, 1]$ , denoted as  $t \sim \mathcal{U}[0, 1]$ .

To support classifier-free guidance, we randomly drop the conditioning label  $o$  with a dropout probability  $p_{\text{drop}}$ . The conditioned label input  $\tilde{o}$  is defined as  $\tilde{o} = \emptyset$  (representing the unconditioned state) with probability  $p_{\text{drop}}$ , and  $\tilde{o} = o$  otherwise. The resulting flow matching objective is formulated as:

$$\mathcal{L}_\pi(\phi) = \mathbb{E}_{(s, a, o) \sim \mathcal{D}_{\text{exp}} \cup \mathcal{D}_{\text{nexp}}, t \sim \mathcal{U}[0, 1], a_0 \sim \mathcal{N}(0, I)} \left[ \|v_\phi(a_t, t, s, \tilde{o}) - (a - a_0)\|^2 \right], \quad (9)$$

where  $s$  denotes the policy’s observation input of the current frame  $f_{k,i}$ .

**CFGRL Inference.** During deployment, we generate guided actions by integrating the ODE defined by the velocity field. Starting from pure Gaussian noise  $a_0 \sim \mathcal{N}(0, I)$  at  $t = 0$ , we perform  $T$  fixed-size Euler integration steps with step size  $\Delta t = 1/T$ :

$$a_{t+\Delta t} = a_t + \Delta t \cdot v_{\text{cfg}}(a_t, t, s), \quad (10)$$

where the guided velocity field  $v_{\text{cfg}}(a_t, t, s)$  is computed by extrapolating towards the high-optimality label ( $o = 1$ ):

$$v_{\text{cfg}}(a_t, t, s) = v_\phi(a_t, t, s, \emptyset) + w \left[ v_\phi(a_t, t, s, o=1) - v_\phi(a_t, t, s, \emptyset) \right]. \quad (11)$$

Here,  $w \geq 1$  is the guidance scale that controls the strength of the optimality bias. Setting  $w > 1$  actively steers the generated actions toward trajectories classified as highly progress-advancing by STEAM. After  $T$  integration steps, the resulting endpoint  $a_1$  is sent to the robot controller as the target action chunk.

## B Pseudocode

We provide the formal algorithmic procedures for our framework. Specifically, Algorithm 1 details the self-supervised training process for the STEAM advantage predictor ensemble. Algorithm 2 describes the policy learning pipeline, showing how learned advantages are translated into binary optimality labels to guide a classifier-free flow-matching policy.

---

**Algorithm 1** Training STEAM

---

**Input:** Expert dataset  $\mathcal{D}_{\text{exp}}$ , ensemble size  $M$ , bin count  $N$ , max offset  $k_{\text{max}}$ , reference length  $L_{\text{max}}$ , offset bound  $\tilde{\Delta}_{\text{max}}$   
**Output:** Trained ensemble  $\Theta = \{\theta_m\}_{m=1}^M$

- 1: **for**  $m = 1$  **to**  $M$  **do**
- 2:     Initialize  $\theta_m$  with an independent random seed
- 3:     **repeat**
- 4:         Sample expert episode  $\tau_k = (f_{k,1}, \dots, f_{k,L_{\tau_k}})$  with instruction  $\ell$  from  $\mathcal{D}_{\text{exp}}$
- 5:         Sample offset  $\Delta \sim \text{Uniform}\{\pm 1, \dots, \pm k_{\text{max}}\}$
- 6:         Sample frame index  $i \sim \text{Uniform}\{i' : 1 \leq i' \leq L_{\tau_k}, 1 \leq i' + \Delta \leq L_{\tau_k}\}$ ; set  $j \leftarrow i + \Delta$
- 7:          $\tilde{\Delta}_{\tau_k}(i, j) \leftarrow (j - i) \cdot L_{\text{max}} / L_{\tau_k}$   $\triangleright$  Eq. (2)
- 8:          $y \leftarrow \tilde{\Delta}_{\tau_k}^{\mathcal{B}}(i, j)$   $\triangleright$  one-hot bin of  $\text{clip}(\tilde{\Delta}_{\tau_k}(i, j), \pm \tilde{\Delta}_{\text{max}})$
- 9:          $\mathcal{L} \leftarrow H(y, p_{\theta_m}(\Delta \mid f_{k,i}, f_{k,j}, \ell))$   $\triangleright$  Eq. (3)
- 10:         Update  $\theta_m$  via gradient step on  $\mathcal{L}$
- 11:     **until** convergence
- 12: **end for**
- 13: **return**  $\Theta = \{\theta_m\}_{m=1}^M$

---

## C Task Descriptions and Training Data Composition

To comprehensively evaluate STEAM across different manipulation horizons, contact patterns, and coordination requirements, we design four real-world tasks. The detailed training dataset composition and step-by-step procedures for each task are described below:

### C.1 Towel Folding

This long-horizon task utilizes an ARX dual-arm robot to fold a towel. A successful execution requires completing all five consecutive stages without severe fabric misfolding. The dataset collected for this task includes 240 expert demonstrations, 125 autonomous rollouts, and 20 human correction episodes. The folding sequence is divided into five stages:

- **Stage 1 (Pick up):** The robot arms reach down and grasp the corners of the towel on the table.
- **Stage 2 (Flatten):** The arms lift and stretch the towel to flatten the fabric, preparing it for folding.
- **Stage 3 (First fold):** The robot executes the first longitudinal fold.
- **Stage 4 (Second fold):** The robot executes the second fold, reducing the towel’s width.
- **Stage 5 (Third fold):** The robot performs the final fold to complete the folding sequence.

### C.2 Chip Checkout

This dual-arm coordination task simulates a retail checkout process, where two bags of chips must be scanned and bagged sequentially. The training dataset for this task includes 200 expert demonstrations, 63 autonomous rollouts, and 20 human correction episodes. The task involves 8 stages:

- **Stage 1 (Pick up first bag):** The left arm reaches and grasps the first bag of chips from the counter.
- **Stage 2 (Scan first bag):** The left arm moves the chip bag to the barcode scanner and performs the scanning action.
- **Stage 3 (Handover first bag):** The left arm coordinates with the right arm to handover the first chip bag.

---

**Algorithm 2** CFGRL Policy Learning with STEAM

---

**Input:** Trained STEAM ensemble  $\Theta = \{\theta_m\}_{m=1}^M$ , expert dataset  $\mathcal{D}_{\text{exp}}$ , non-expert dataset  $\mathcal{D}_{\text{nexp}}$ , policy lookahead  $H$ , quantile level  $\varpi_q^{\text{exp}}$  and  $\varpi_q^{\text{non-exp}}$ , conditioning dropout  $p_{\text{drop}}$

**Output:** Trained flow-matching policy  $v_\phi$

**Stage 1: label data with STEAM**

- 1: **for** each  $(s_t, a_t, s_{t+H}) \in \mathcal{D}_{\text{exp}} \cup \mathcal{D}_{\text{nexp}}$  **do**
- 2:      $A_{\text{STEAM}}(s_t, s_{t+H}) \leftarrow \min_{m=1, \dots, M} A(s_t, s_{t+H}; \theta_m)$  ▷ Eqs. (4), (5)
- 3: **end for**
- 4:  $\varpi_q^{\text{exp}} \leftarrow q\text{-quantile of } \{A_{\text{STEAM}}(s_t, s_{t+H})\} \text{ over } \mathcal{D}_{\text{exp}}$
- 5:  $\varpi_q^{\text{non-exp}} \leftarrow q\text{-quantile of } \{A_{\text{STEAM}}(s_t, s_{t+H})\} \text{ over } \mathcal{D}_{\text{nexp}}$
- 6: **for** each  $(s_t, a_t, s_{t+H}) \in \mathcal{D}_{\text{exp}} \cup \mathcal{D}_{\text{nexp}}$  **do**
- 7:     **if**  $(s_t, a_t, s_{t+H}) \in \mathcal{D}_{\text{exp}}$  **then**
- 8:          $o_t \leftarrow \mathbf{1}[A_{\text{STEAM}}(s_t, s_{t+H}) \geq \varpi_q^{\text{exp}}]$
- 9:     **else**
- 10:          $o_t \leftarrow \mathbf{1}[A_{\text{STEAM}}(s_t, s_{t+H}) \geq \varpi_q^{\text{non-exp}}]$
- 11:     **end if**
- 12: **end for**
- 13:  $\mathcal{D}_o \leftarrow \{(s_t, a_t, o_t)\}$  ▷ Eq. (7)

**Stage 2: classifier-free guidance policy training**

- 14: Initialize flow-matching policy parameters  $\phi$
- 15: **repeat**
- 16:     Sample  $(s, a, o) \sim \mathcal{D}_o$
- 17:     Sample flow time  $t \sim \mathcal{U}[0, 1]$  and noise  $a_0 \sim \mathcal{N}(0, I)$
- 18:      $a_t \leftarrow (1 - t) a_0 + t a$
- 19:      $\tilde{o} \leftarrow \emptyset$  with probability  $p_{\text{drop}}$ , otherwise  $\tilde{o} \leftarrow o$
- 20:      $\mathcal{L}_\pi \leftarrow \|v_\phi(a_t, t, s, \tilde{o}) - (a - a_0)\|^2$  ▷ Eq. (9)
- 21:     Update  $\phi$  via gradient step on  $\mathcal{L}_\pi$
- 22: **until** convergence
- 23: **return**  $v_\phi$

---

- 410     • **Stage 4 (Bag first bag):** The right arm receives the chip bag and places it into the shopping
- 411         bag.
- 412     • **Stage 5 (Pick up second bag):** The left arm reaches and grasps the second bag of chips.
- 413     • **Stage 6 (Scan second bag):** The left arm moves the second chip bag to the scanner for
- 414         barcode recognition.
- 415     • **Stage 7 (Handover second bag):** The left arm handovers the second chip bag to the right
- 416         arm.
- 417     • **Stage 8 (Bag second bag):** The right arm receives the second chip bag and completes the
- 418         bagging process.

### 419 C.3 Cola Restocking

420 This task requires the dual-arm robot to transfer a cola bottle from a crate to a shelf, which de-  
421 mands active collision avoidance. The training dataset consists of 89 expert demonstrations and 49  
422 autonomous rollouts. The task is divided into 4 stages:

- 423     • **Stage 1 (Pick up):** The left arm grasps a cola bottle from the inventory crate.
- 424     • **Stage 2 (Camera adjustment):** The left arm rotates its wrist to adjust the wrist-camera's
- 425         viewpoint, preventing potential hardware and camera collisions between the two arms.
- 426     • **Stage 3 (Handover):** The left arm handovers the cola bottle to the right arm.
- 427     • **Stage 4 (Place):** The right arm moves the cola bottle and places it stably onto the target
- 428         shelf.

## C.4 Pick-and-Place

This short-horizon task employs a single Franka arm to relocate objects between two plates. The training dataset contains 50 expert demonstrations and 594 autonomous rollouts. The task is divided into 2 stages:

- **Stage 1 (Grasp):** The gripper descends to grasp a target object from the left plate.
- **Stage 2 (Place):** The arm transfers and places the object onto the designated position on the right plate.

## D Hyperparameters

Table 4 provides the comprehensive hyperparameter configurations used for the STEAM advantage predictor and the CFGRL policy training across all four real-world robot manipulation tasks.

Table 4: Hyperparameter configurations for STEAM advantage prediction and CFGRL policy optimization.

| Hyperparameter                                                            | Towel Folding      | Chip Checkout      | Cola Restocking    | Pick-and-Place     |
|---------------------------------------------------------------------------|--------------------|--------------------|--------------------|--------------------|
| Maximum offset                                                            | 32                 | 32                 | 32                 | 16                 |
| Number of bins $N$                                                        | 32                 | 32                 | 32                 | 16                 |
| Ensemble size $M$                                                         | 3                  | 3                  | 3                  | 3                  |
| Advantage lookahead $H$                                                   | 32                 | 32                 | 32                 | 16                 |
| $q$ -quantiles over $\mathcal{D}_{\text{exp}} \varpi_q^{\text{exp}}$      | 0.8 (top 80%)      | 0.8 (top 80%)      | 0.8 (top 80%)      | 0.8 (top 80%)      |
| $q$ -quantiles over $\mathcal{D}_{\text{nexp}} \varpi_q^{\text{non-exp}}$ | 0.3 (top 30%)      | 0.3 (top 30%)      | 0.3 (top 30%)      | 0.3 (top 30%)      |
| Conditioning dropout $p_{\text{drop}}$                                    | 0.1                | 0.1                | 0.1                | 0.1                |
| Guidance scale $w$                                                        | 2.5                | 2.5                | 2.5                | 2.5                |
| Learning rate                                                             | $5 \times 10^{-5}$ | $5 \times 10^{-5}$ | $5 \times 10^{-5}$ | $5 \times 10^{-5}$ |
| Training steps                                                            | 30,000             | 30,000             | 30,000             | 30,000             |
| Batch size                                                                | 512                | 512                | 512                | 512                |

## E Additional Advantage Visualizations

For completeness, we extend the qualitative analysis presented in Section 4 (Figure 4) to the remaining three manipulation tasks. Figures 8, 9, and 10 illustrate the frame-level  $A_{\text{STEAM}}$  advantage curves across various episode types for the chip checkout, cola restocking, and pick-and-place tasks, respectively.

### E.1 Chip Checkout

For the **chip checkout** task (Figure 8), the advantage curves accurately capture the complex dual-arm coordination and timing. In the *expert demonstration*, a brief grasping failure by the right arm during the handover phase (around frames 200–300) causes a temporary drop in the advantage; once a successful retry is executed, the advantage recovers to high values. In the *successful rollout*, the overall execution speed is visibly slower, particularly during the handover phase (indicated by the dark blue region), leading to a more fluctuating advantage curve. The *failed rollout* exhibits a persistent decline, with the advantage remaining extremely low after frame 400. In the *human-correction episode*, the policy stagnates around frame 800 and fails to grasp the second chip bag, causing the advantage to dip significantly into negative values. Following human takeover and corrective action, the task resumes progress, and the advantage recovers.

### E.2 Cola Restocking

For the **cola restocking** task (Figure 9), we observe similar temporal sensitivity to coordination and placement errors. In the *expert demonstration*, the robot fails to place the cola bottle onto the narrow



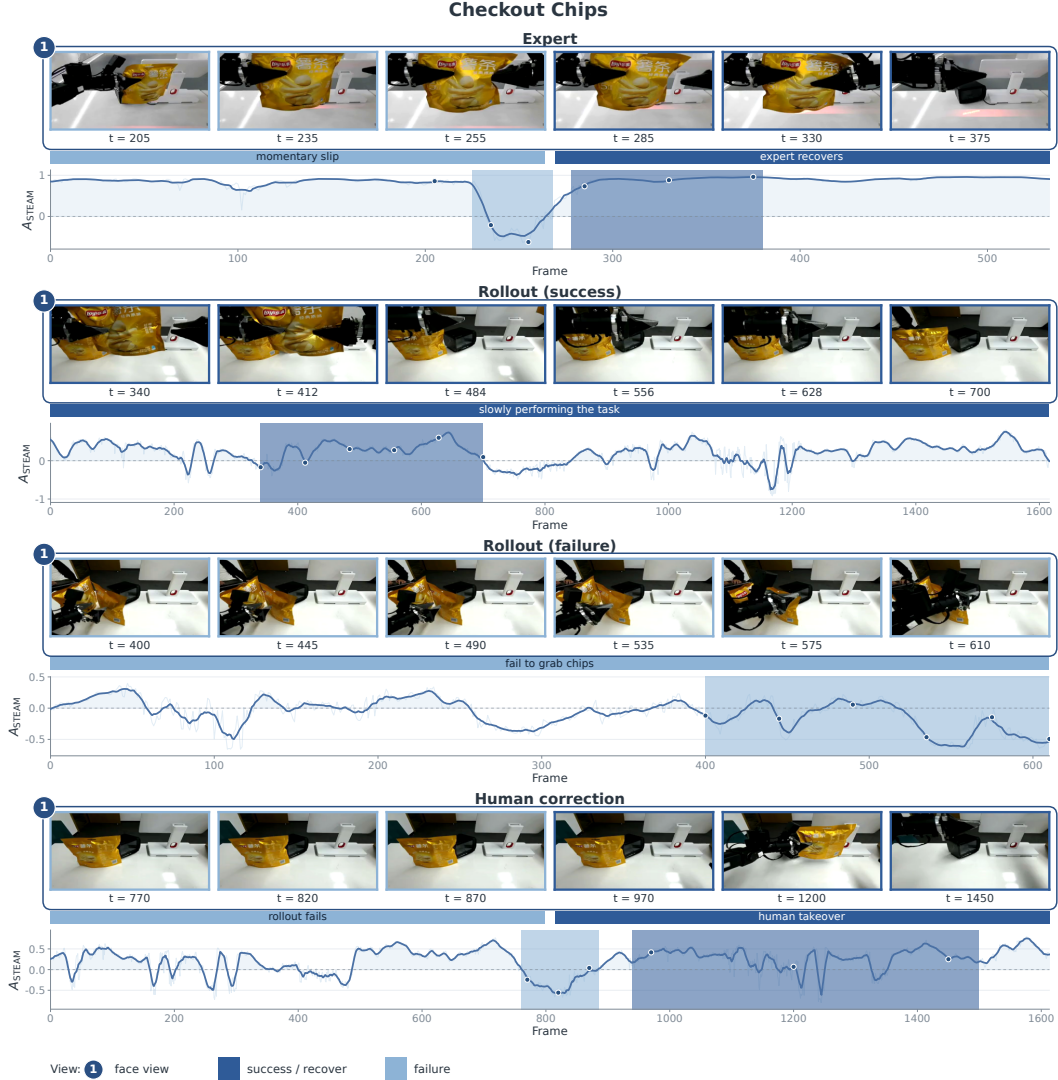


Figure 8: **Chip checkout advantage curves.** Expert, successful-rollout, failed-rollout, and human-correction episodes.

shelf due to initial misalignment between frames 400 and 600, causing a temporary advantage drop. After a successful realignment and retry after frame 600, the advantage rises back to a high level. In the *successful rollout*, the right arm makes multiple consecutive attempts to grasp the cola during the handover phase, leading to slower progress and a corresponding decline in the advantage curve. In the *failed rollout*, the advantage drops and remains consistently low after frame 200, reflecting unrecoverable policy errors.

### E.3 Pick-and-Place

For the **pick-and-place** task (Figure 10), which has a much shorter horizon, the advantage curves remain highly informative. The *expert demonstration* shows smooth, proficient execution, maintaining a high advantage near +1 throughout. In the *successful rollout*, the robot makes multiple unsuccessful attempts to grasp the target object (a toy duck) between frames 40 and 100, pushing the advantage below zero. After a successful grasp after frame 100, the advantage recovers to nearly +1. In the *failed rollout*, the advantage is initially high (frames 0–50) as the gripper approaches the duck, but plunges below zero once the robot repeatedly fails to grasp the object.

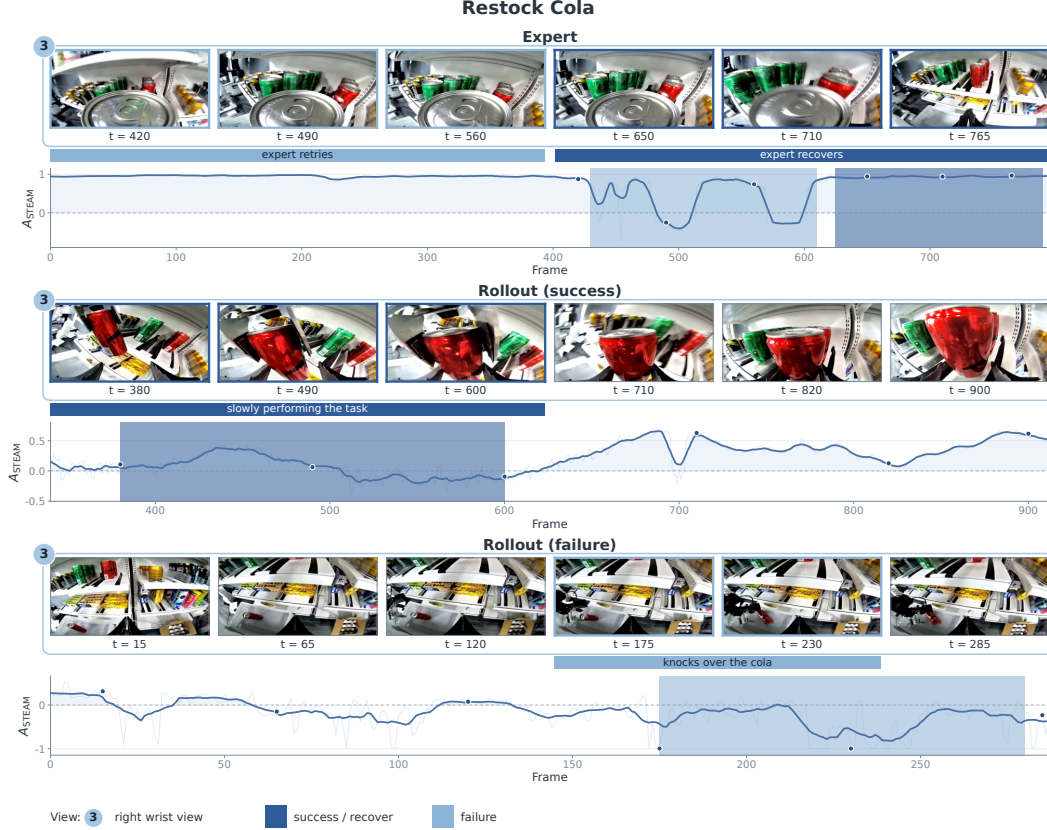


Figure 9: Cola restocking advantage curves. Expert, successful-rollout, and failed-rollout episodes.

#### 472 E.4 Effectiveness of Conservative Ensemble Aggregation

473 To demonstrate why a conservative ensemble strategy is essential for robust advantage estimation,  
 474 we visualize the individual estimates from each ensemble member alongside their aggregated min-  
 475 imum in Figure 11. Specifically, we plot the advantage curves of three independent predictors ( $\theta_1$ ,  
 476  $\theta_2$ , and  $\theta_3$ ) and the final aggregated  $A_{STEAM}$  advantage for a representative towel-folding rollout.

477 As shown in Figure 11, the robot performs a retry during the final folding stage between frames 1200  
 478 and 1400. During this regression phase, the individual predictors exhibit highly divergent behaviors.  
 479 **Ensemble 1 (blue)** and **Ensemble 2 (orange)** correctly identify the regression, with their estimated  
 480 advantages dropping significantly (with Ensemble 2 dipping below 0.0). Conversely, **Ensemble 3**  
 481 **(green)** suffers from severe out-of-distribution overestimation, remaining confidently high (above  
 482 0.8) and falsely signaling that successful progress is being maintained.

483 This divergence highlights the vulnerability of using a single model, which can easily become  
 484 overconfident on atypical or regressive behaviors. By applying our conservative aggregation strat-  
 485 egy—taking the pointwise minimum across the ensemble (Eq. (5))—the final  $A_{STEAM}$  advantage  
 486 (represented by the bold black curve) successfully suppresses the false-positive signal from Ensem-  
 487 ble 3. The resulting aggregated curve correctly drops below  $-0.5$  during the retry window. This  
 488 qualitative analysis confirms that conservative ensemble aggregation is vital for mitigating overesti-  
 489 mation and ensuring stable policy guidance.

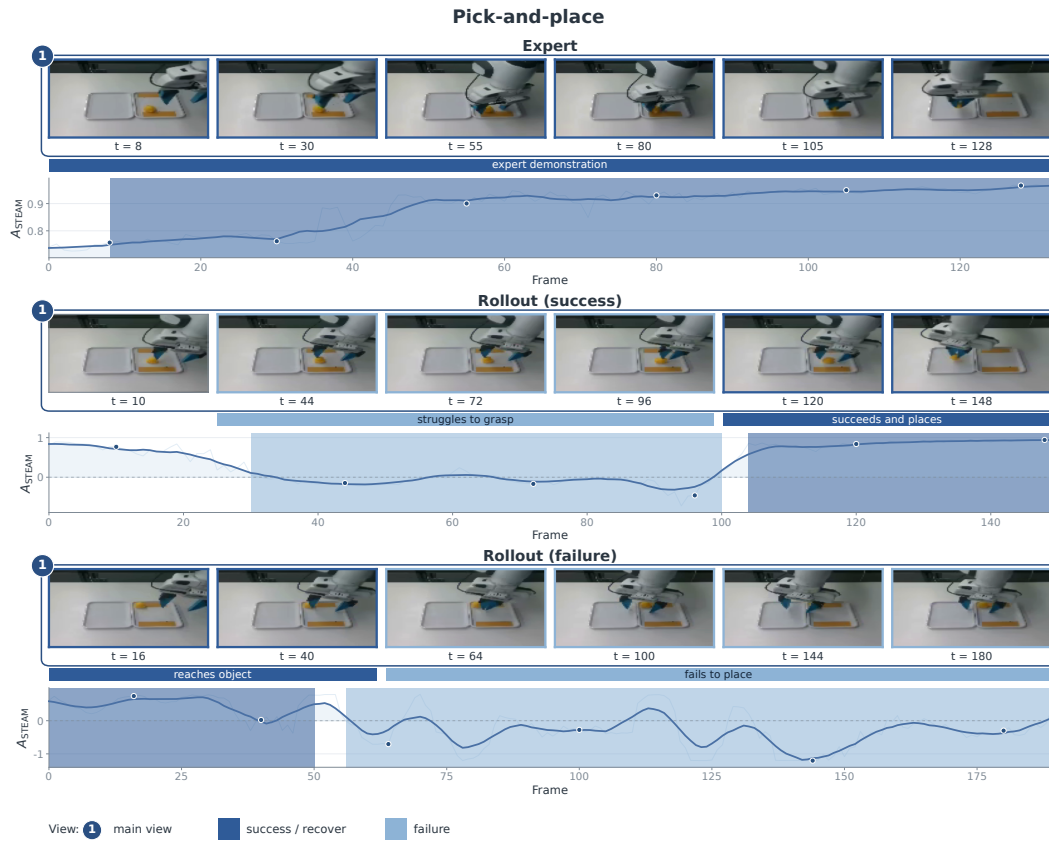


Figure 10: **Pick-and-place advantage curves.** Expert, successful-rollout, and failed-rollout episodes.

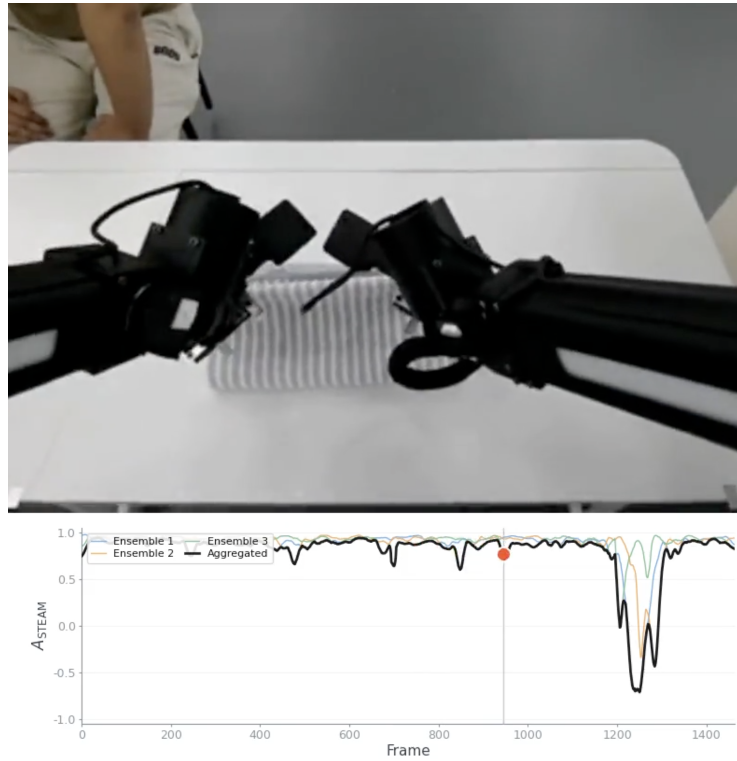


Figure 11: **Visualization of individual ensemble predictions vs. aggregated STEAM advantage.** During the retry of the final towel-folding step (between frames 1200 and 1400), the green curve (Ensemble 3) severely overestimates the advantage. By taking the minimum across the ensemble, the final aggregated STEAM advantage (black curve) successfully suppresses this false positive.